

SURVEY

Application of Machine Learning in FPGA EDA Tool Development

PINGAKSHYA GOSWAMI¹, (Student Member, IEEE),

AND DINESH BHATIA², (Senior Member, IEEE)

Department of Electrical and Computer Engineering, The University at Texas at Dallas, Richardson, TX 75080, USA

Corresponding author: Pingakshya Goswami (pingakshya.goswami@utdallas.edu)

ABSTRACT With the recent advances in hardware technologies like advanced CPUs and GPUs and the large availability of open-source libraries, machine learning has penetrated various domains, including Electronics Design Automation (EDA). EDA consists of multiple stages, from high-level synthesis and logic synthesis to placement and routing. Traditionally, estimating resources and areas from one level of design abstraction to the next level uses mathematical, statistical, and analytical approaches. However, as the technology node decreases and the number of cells inside the chip increases, the traditional estimation methods fail to correlate with the actual post-route values. Machine-learning (ML) based methodologies pave a strong path towards accurately estimating post-route values. In this paper, we present a comprehensive survey of the existing literature in the ML application field in EDA, emphasizing FPGA design automation tools. We discuss how ML is applied in different stages to predict congestion, power, performance, and area (PPA), both for High-Level Synthesis (HLS) and Register Transfer Level (RTL)-based FPGA designs, application of design space exploration and application in Computer-Aided Design (CAD) tool parameter settings to optimize timing and area requirements. Reinforcement learning is widely applied in both FPGA and ASIC physical design flow, a topic of discussion in this paper. We also discuss various ML models like classical regression and classification ML, convolution neural networks, reinforcement learning, and graph convolution network and their application in EDA.

INDEX TERMS Machine learning, electronic design automation, FPGA.

ACRONYMS

ADRS	Average Distance to Reference Set.
ANN	Artificial Neural Network.
CAD	Computer Added Design.
CDFG	Control and Dataflow Graph.
CNN	convolution Neural Network.
DRC	Design Rules Check.
DSE	Design Space Exploration.
EDA	Electronics Design Automation.
GA	Genetic Algorithm.
GAN	Generative Adversarial Network.
GCN	Graph convolution Network.
HLS	High Level Synthesis.
HPWL	Half Perimeter Wirelength.
IC	Integrated Circuit.

IR	Intermediate Representation.
MAPE	Mean Absolute Percentage Error.
MARS	Multivariate Adaptive Regression Splines.
ML	Machine Learning.
MLP	Multi Layer Perceptron.
PPA	Power Performance and Area.
RL	Reinforcement Learning.
RTL	Register Transfer Language.
SA	Simulated Annealing.
TNS	Total Negative Slack.
XGB	Gradient Boost Regression.

I. INTRODUCTION

As the technology node size decreases due to Moore's Law [1], and Dennard's scaling [2], more and more transistors are fitted per unit area inside the integrated circuit (IC) than ever before. With the decrease in technology size, the tools used to design the ICs are becoming more complex.

The associate editor coordinating the review of this manuscript and approving it for publication was Mauro Tucci³.

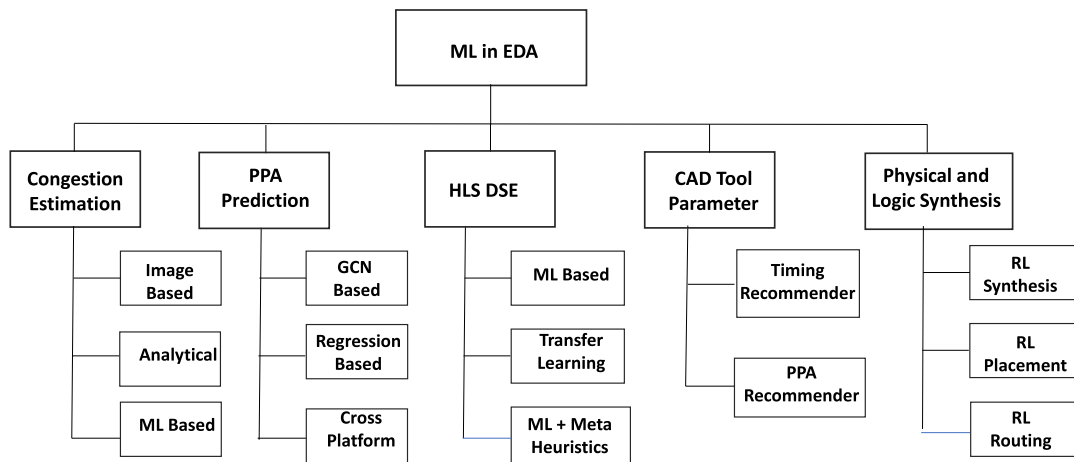


FIGURE 1. Different categories in which ML is applied in FPGA EDA Flow.

Electronic Design Automation (EDA) is the branch of science or technology that deals with the tools used to design integrated circuits. IC design is a highly complex process consisting of multiple stages with an average turnaround time ranging from a few days to a few years. While designing an IC, it is always a good practice to estimate the power, performance, area, congestion, and wirelength of the design down the flow in an early stage. Traditionally, this is done using statistical, mathematical, or analytical methods [3], [4], [5]. Although the analytical and mathematical models get pretty accurate estimates, these methods are computationally expensive and consume a lot of time (few hours) to perform basic estimations like congestion [3], [4] and wirelength [6]. These methods generally solve every problem from scratch and do not utilize any experience or knowledge.

On the other side of the technology world, another domain is emerging pretty fast and has shown its application in almost every aspect of human life known as Machine Learning (ML). ML has seen its applications in various domains like image recognition [7], [8], natural language processing [9], [10], [11], audio and signal processing [12], [13], [14], etc. This is because the hardware used for training the models, like GPUs, has progressed significantly with multiple processors fabricated inside one die along with easy availability of open source training frameworks like Tensorflow, Scikitlearn, and Pytorch [15], [16], [17]. ML has also penetrated the EDA domain in the last five to six years. All the problems that were previously solved using analytical methods [3], [4], [5], [6], [18] are now reframed as ML problems. The issues that took a few minutes to hours using analytical or mathematical methods can now be solved using ML-based pre-trained models within a few seconds. Machine learning has found its application in almost all stages of IC designs, which include timing, power, and area estimation; routing congestion estimation, both using classical ML and deep learning methods; design space exploration; CAD tool parameter tuning; Wirelength (WL) estimation and lithographic hotspot detection. For this

paper, we have surveyed the literature in the ML application domain in EDA tool design and optimization, focusing on FPGA EDA tools. We have clustered the application in the following five categories:

- i. Power, performance and area prediction in RTL and HLS Designs
- ii. Design Space Exploration (DSE)
- iii. CAD tool optimization using ML
- iv. Congestion estimation in HLS and RTL designs
- v. ML in FPGA Physical Design Flow

In Fig. 1, we have shown how ML is applied at various stages of FPGA EDA flow. The figure also shows the various subcategories and ML algorithms applied at each flow stage.

We have discussed both analytical and mathematical-based PPA estimators [19], [20], [21] and ML-based estimators [22], [23], [24], [25], [26], where the design entry is either HLS or RTL input. The prediction models use traditional ML methods like regression and classification and advanced methods like deep learning and graph convolution [26], [27] to estimate the PPA of a circuit. For autotuning and recommendation of CAD tool parameters [28], [29], [30]. Bayesian methods are used to select the hyperparameters of the CAD tools. Yanghua et al. perform the feature selection to reduce the number of FPGA CAD parameters to consider [31] from 80 to 8 features without compromising the quality of results. To reduce the parameter-tuning effort, Kwon et al. [32] propose a CAD tool parameter recommender system that involves learning a collaborative prediction model through tensor decomposition and regression. ML-based design space exploration of HLS designs is discussed in [33], [34], [35], and [36]. Reference [30] uses Bayesian optimization to perform DSE to optimize an HLS-based CNN architecture, while Carloni [35] uses transfer learning to optimize the designs during DSE. Congestion estimation is another problem that machine learning can solve accurately and quickly. In [37] and [38], the congestion estimation of RTL designs on FPGA uses classical regression-based ML modes. Reference [39]

predicts the routing congestion of HLS designs using ML, while [40] and [41] uses convolution neural network(CNN) methods to forecast the congestion on post-placed images of logic netlists.

As shown in Fig. 1, ML can be applied at different stages of the logical and physical design flow, from quality of results (QoR) estimations to hyperparameter selection and design space exploration. In this paper, we comprehensively reviewed how ML is applied in different stages of FPGA physical design flow, along with some insights into future developments and open research areas. The rest of the paper is organized as follows: Section II, we discuss the other works, where application of ML in EDA tool development for both ASICs and FPGAs have been studied; in Section III, we discuss the various ML models applied in EDA tool design, which includes classical regression and classification, CNN, reinforcement learning, transfer learning, and graph convolution network (GCN). From Section IV to Section VIII, we discuss in detail the five categories in which ML is applied for FPGA CAD. Finally, Section IX summarizes the paper's contents and looks into future trends and directions.

II. RELATED SURVEYS IN ML BASED EDA TOOL DEVELOPMENT

There are few survey papers in which different works in the ML application domain in EDA have been studied [42], [43], [44], [45], [46], [47], [48]. Most of these works are either too generic for ASIC [42], [45] or too specific to a particular domain [43], [44], [48]. These papers discuss how ML has been applied to predict or improve the CAD tools' performance to design ASICs. Discussion of ML-based FPGA CAD tools has been neglected in survey papers. Reference [42] is a comprehensive survey in which they have performed a detailed analysis of how ML can be applied to various stages of ASIC design flow. They discussed ML applications for ASIC physical design, power delivery network and IR drop analysis, lithography and mask production, analog IC design, device sizing automation, and verification and testing. As we can see here, in this work, all the predictions are targeted for nanometer scale ASIC physical design flow. They have minimal discussion on the FPGA front, that too on the HLS part, but nothing has been discussed about prediction in various stages of FPGA physical design. In [43], the authors surveyed exclusively on applying graph neural network (GNN), ASIC physical design flow. The paper [43] discusses primarily the theoretical details of GNN, and the discussion on applying GNN on EDA has been minimal. In another work, [44], the authors discussed explicitly the application of ML in analog circuit design. They mentioned the ANN ANN-based analog IC design process, the hybrid analog IC design automation process, and the application of ANN and RL in analog IC manufacturing. In [45], the authors briefly discuss the application of ML in the placement and routing of ASIC designs. There is no mention of FPGAs, HLS, and recent

methods like CNN, GNN, and RL. They only discuss wirelength estimation in placement and Design Rules Check (DRC) hotspot detection in post-routed circuits. Paper [46] and [49] discuss hotspot and DRC violation prediction using ML in ASIC circuits and static timing analysis using ML. These two papers are more of a roadmap paper than a survey paper. These papers discuss in what different domains ML can be applied to future IC designs. Similar to [46] and [47] is a roadmap that discusses performance limitations of traditional compute and storage systems and the systems and infrastructure considerations for performing machine learning at scale. This paper also briefly discusses how ML can be applied to solve functional verification and debug problems. Even though [48] is not a proper survey paper, the authors discuss how RL can be applied to solve various circuit design problems like DRC violations, layout, and routing issues. This work profoundly focuses on using Reinforcement Learning (RL) to solve post-route violation issues.

As the above discussion shows, most existing surveys are highly focused on IC development or EDA tools for ASICs. Even though there are many recent works in which ML has been applied to solve various FPGA physical design problems, their discussion has been limited to the "Related Works" section of the papers only. No comprehensive survey is available that discusses the existing work of ML-based FPGA CAD tool design. To address these loopholes, we created this detailed survey in which we exclusively review various works in which ML has been applied to multiple FPGA CAD tool design stages. We discuss both ML-based HLS tools and ML-based RTL tools for FPGAs. This paper surveys the recent advancements in GNN and RL, which have been widely applied to the synthesis, placement, and routing of FPGA circuits.

III. BASICS OF MACHINE LEARNING

ML algorithms are broadly divided into two classes:

- (i) Supervised Learning
- (ii) Unsupervised Learning

Most of the ML models used in EDA fall under supervised learning, including regression, classification, Convolution Neural Networks, and Graph Convolution Network. The CAD tool optimization or Design Space Exploration tools use Bayesian learning and the Gaussian process to set the hyperparameters of the CAD tools. In Table 1, we have mapped the different EDA applications to the ML models used. In this section, we have discussed the ML algorithms mentioned in column 1 of Table 1 in detail.

- (i) **Linear Regression:** Linear regression is a method to find relationship between two continuous variables; one is independent, and the other is dependent. Linear regression tries to create a statistical relationship that may not always be deterministic. Linear regression tries to obtain a straight line that shows the relationship between the independent and the dependent variable. For most of the linear regression models, the error

TABLE 1. ML models and their applications in EDA flow.

ML Model	Application	Reference
Regression	- Power Prediction - HLS Area and Timing - Congestion - DSE - Cross Platform - Performance Estimation	[22], [25], [37], [39], [36], [38], [50], [51], [52]
Analytical	Power, Performance and Latency estimation	[19], [21] [20]
GCN	HLS QoR Estimation	[26], [27]
Transfer Learning	DSE	[35], [53]
Reinforcement Learning	DSE	[26]
Decision Tree	DSE	[33], [34]
GAN	Routing congestion	[40], [41]
Bayesian Opt	CAD tool parameter	[29], [30]
Classification	CAD tool parameter	[28], [54], [55]
Reinforcement Learning	Place and Route	[56]–[59] [57], [58], [60], [61]
Graph Neural Network	Power	[62], [63], [64]

is measured using the least squared error. A linear regression line has an equation of $Y = a + bX$, where X is the explanatory variable and Y is the dependent variable. The slope of the line is b , and a is the intercept, which is determined during model training.

(ii) **Random Forest Regression(RF)**: Random forest is a supervised learning method that uses ensemble methods for learning. Ensemble learning is a technique that combines multiple weak learners to create a strong learner. In random forests, these weak learners are decision trees. The outputs from the individual learners are averaged to generate the final output. Random forest belongs to the bagging class of learners where, during training, data samples are selected at random without replacement. Bagging makes each model run independently and then aggregates the outputs at the end without preference for any model. Because of the introduction of the bagging method, the chances of overfitting are less in a random forest. Random forest is a very fast training method because each tree can be trained in parallel, and the inputs and outputs of each tree are not related to one another. To summarize, Random Forest Algorithm merges the output of multiple Decision Trees to generate the final output.

(iii) **Multivariate Adaptive Regression Splines (MARS)**: Multivariate adaptive regression splines [65], an algorithm that automatically creates a piecewise linear model which creates a nonlinear model by combining multiple small linear functions known as steps. In MARS, non-linearity is introduced by using step functions. There are no polynomial terms in the MARS equation. Equation 1 shows the linear stepwise equation:

$$y_i = \beta_0 + \beta_1 C_1(x_i) + \beta_2 C_2(x_i) + \dots + \beta_d C_d(x_i) + \epsilon_i, \quad (1)$$

MARS is an adaptive procedure for regression and is well-suited for high-dimensional problems (i.e., many inputs). The bends in the step functions are known

as “knots”. If there are multiple knots present in the MARS equation, it may lead to overfitting.

- (iv) **Multi Layer Perceptron based Regression (MLP)**: MLP-based regression models comprise multiple perceptrons known as neurons. This type of model falls into the feedforward class of artificial neural networks. Generally, Artificial Neural Networks (ANN)s are used for classification. But they can be used for regression, too. MLP-based models are trained using a backpropagation algorithm. Each network consists of an input layer of neurons, a few hidden layers, and an output layer. The output of each function is linear. To make the output non-linear and emulate the real-world behavior, activation functions like “*tanh*”, “*RELU*” and “*sigmoid*” are added.
- (v) **Gradient Boost Regression (XGB)**: Similar to random forest regression, Gradient boost regression is also an ensemble class of ML model. Boosting is a method of converting weak learners into strong learners. Unlike random forest regression, where we use fully grown trees, in XGB, the trees are weak learners; hence, shallow trees can also be used for fast learning. In boosting, each new tree is fit on a modified version of the original data set. XGB uses multiple trees in sequence, which fixes the error of the previous tree. The XGB uses a gradient decent algorithm on the loss function to minimize the error generated by the previous trees. The XGB method keeps adding trees one after another iteratively until there is no improvement in the loss. The loss function used is either sum of squared error (SSE) or mean square error (MSE) [66].
- (vi) **Graph Convolution Network(GCN)**: With the wide application of deep learning, neural networks are an effective and efficient model for tasks like classification and regression. However, neural networks like ANN and CNN only take vectors or tensors as input data, which makes it difficult to work on graphs. Defferrard et al. generalize convolutional neural networks from regular grids (e.g., images) to general

graphs via graph convolutional filters [67]. Later, Kipf and Welling simplify the convolutional operator and propose a graph convolutional network [68]. Graph learning techniques can be categorized into transductive and inductive settings. Under the transductive setting, the embedding of each node is directly optimized, and thus, the training process requires seeing all the nodes. The original GCN [67] is one kind of transductive approach. The inductive approaches learn a general rule from the training graphs through sample and aggregation of structural information and node attributes. The learned model can be applied to unseen data.

- (vii) **Reinforcement Learning(RL):** Reinforcement learning is an ML algorithm in which the machine tries to learn by itself. The machine repeatedly uses trial and error to learn from its own experience. To program the machine, the programmer sets rewards for correct actions and penalizes the machine for wrong actions. The goal of the machine is to maximize the rewards. Unlike supervised learning, the training data of reinforcement learning have no labels. By leveraging the power of search and many trials, reinforcement learning is currently the most effective way to hint at a machine's creativity [69], [70].
- (viii) **Bayesian Optimization:** Bayesian optimization is an optimization technique that uses Bayes Theorem to find the global maxima or global minima of a non-convex, non-linear, multi-dimensional function. It is highly suitable for optimizing functions consisting of less than twenty dimensions. Bayesian optimization consists of two parts: a Surrogate Function, which uses the Bayesian model to model the objective function, and an Acquisition Function to decide what dimension value to sample next. Gaussian Process Regression is an example of an acquisition function. Bayesian optimization is used in hyperparameter tuning of ML models and hyperparameter of CAD tools [29].

In the rest of the paper, we will discuss the various applications of ML in EDA flow.

IV. ML IN ROUTING CONGESTION ESTIMATION

Detailed routing is the most time-consuming stage in VLSI or FPGA design flow. Generally, the routing in VLSI is divided into two stages: a) Global Routing and b) Detailed Routing. Congestions generally occur in the detailed routing stage, which results in a violation of DRC rules, and the circuit sometimes becomes unroutable. Routing congestion can be measured accurately only after detailed routing has been completed, which takes a lot of time. Because of these reasons, predicting the approximate value of routing congestion before the actual detailed routing stage can save both time and energy for the designers. Since global routing-based congestion estimation cannot be estimated accurately for smaller technology nodes, researchers have

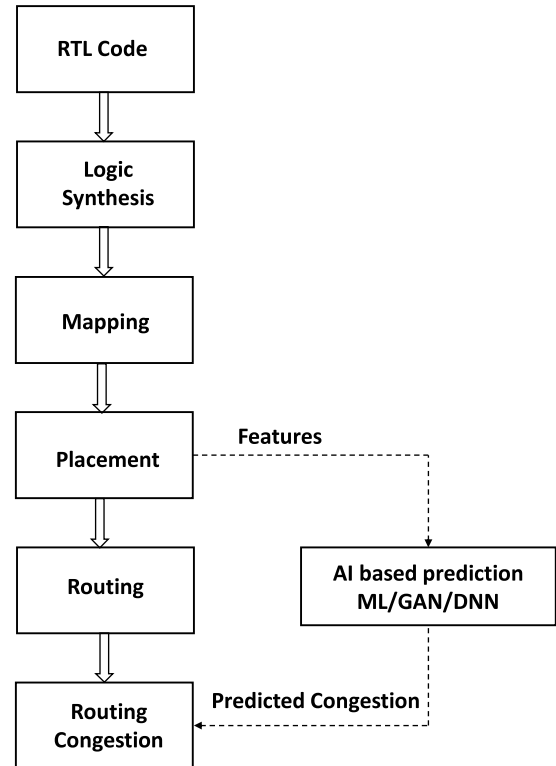


FIGURE 2. Conventional FPGA routing flow and the AI based flow shown in dotted line on right.

developed ML-based congestion estimation methods for FPGAs and ASICs. They have used traditional regression models on RTL netlists [37], [38] and HLS designs [39]. Other researchers have addressed the FPGA congestion prediction problem as Generative Adversarial Network (GAN) and CNNs [27], [41]. In these models, the authors apply image-based GAN and CNN algorithms to detect congestion hotspots on the 2D images of post-placed netlists. This section discusses the ML-based works to predict congestion on FPGA netlists. In Fig. 2, we have shown a conventional FPGA/ASIC design flow starting from the RTL code to the routing congestion map level. On the right side, we have shown how ML can be applied right after the placement stage to get a quick estimate of the post-route congestion value. The applied ML model can be traditional regression/classification methods or image-based models like CNN and GAN (generative adversarial network).

A. ROUTING CONGESTION PREDICTION USING CLASSICAL ML MODELS

There are three recent works in which FPGA routing congestion prediction problem has been addressed as a regression problem [37], [38], [71]. The authors divided the FPGA architectures into global routing cells called *gcells*, which consist of one CLB and one switch box, and predict the routing congestion in each *gcell*. In [37] and [38], the authors considered various intrinsic properties of post-placed FPGA netlists like pin density, Wirelength Per

TABLE 2. Classification of ML algorithms and their application in congestion estimation.

Paper	[37]	[38]	[39]	[71]	[40]	[41]
Design Entry	RTL	RTL	HLS	RTL	RTL	RTL
ML Category	Regression	Regression	Regression	Regression	GAN	GAN
MAPE %	4.62	12.8	10.57	3.61	14.6	10.7
ML Model Used	MLP	Global Linear	XGB	RF	GAN	CGAN
Feature source	Placed NL	Placed NL	LLVM IR	Placed NL	Image encoded	Image encoded
Dataset	ISPD2016	ISPD2016	Image processing IP	ISPD2016	Custom VPR IP	ISPD2016

Area (WLPA), Net Cut Per Region (NCPR) etc., as their features to the regression model. Goswami et al. [71] considered a total of nine features for their model, which include LUT and Flipflop utilization, incoming/outgoing and fully local nets in each *gcells* and relative location of each *gcell* inside the FPGA device. All three works tested and verified their models on ISPD 2016 [72] benchmarks. While [37] and [38] divided their dataset into 70% train and 30% test dataset, [71] predicted on individual benchmarks. To create their models, the three works used conventional regression methods like linear regression, random forest, and gradient boost.

B. ROUTING CONGESTION PREDICTION IN HLS DESIGN

One of the major limitations of the current commercial HLS tools is that it cannot estimate routing estimation, even inaccurately. In [39], Zhao et al. address this issue by creating an ML model that predicts routing congestion after the C-synthesis stage, i.e., before actual RTL synthesis. If high congestion is detected for a certain portion of the code, they suggested methods to modify the high-level C/C++ code, reducing congestion after routing. To do so, they converted the HLS code into LLVM-IR [73] and backtracked each of the instructions on the IR code to the actual layout after routing. The back tracing is done in the following way:

Congested CLB → gate level netlist → RTL code → IR code → HLS code

Once the source of the instruction for the congested CLB is determined in the HLS code, features are extracted to create a regression model. The regression model is later used to predict the post-route congestion from the high-level C/C++ and LLVM IR codes.

C. ROUTING CONGESTION PREDICTION USING GAN BASED METHOD

The work described in [40] and [41] addressed the routing congestion estimation problem as a conditional generative adversarial network (CGAN) problem. In both works, the authors transfer the routing congestion problem in large-scale FPGAs to an image-to-image problem and then use conditional GAN to solve it. In [41], proposed a new placement-based routing congestion prediction approach for large-scale FPGA designs. The proposed approach adopts a new CGAN model, namely *pix2pixHD*, which performs high-definition (HD) image translations for large high-resolution images. With HD image translation, this

approach can achieve high prediction accuracy for large FPGA designs while relying on well-engineered features that encode the placement and connectivity information for large-scale designs. The main contributions of [40] include selecting features only from post-placed netlist image, estimation of the routing channel utilization by forecasting the full congestion heat map instead of hotspots only, and integration with placement tool to estimate routing congestion on the fly during placement. Both tools can be easily integrated with open-source academic placers like UTPPlace and GPlace [74], [75] and achieve very high accuracy.

D. SUMMARY OF ROUTING CONGESTION WORK AND FUTURE DIRECTIONS

The application of ML to solve congestion estimation can be broadly classified into two classes: (a) Using traditional regression methods and (b) Using image-based GAN methods. Also, the design entry to the tool can be either RTL-based or HLS-based. In Table 2, we have summarized all the ML-based congestion estimation tools for FPGAs and the models used for prediction. In most cases, the performance of the post-route congestion estimation tool is measured in terms of mean absolute percentage error (MAPE). In row 4 of Table 2, we present the average error reported in each of the six works. Since post-route congestion is estimated after the placement stage of the FPGA physical design process, for RTL-based designs, features are extracted from the post-placement netlists. Even for GAN-based designs, placement netlists are converted to images, and features are extracted from those images. For example, in [41], encoded feature maps in high-definition images are created for pin density vertical and horizontal routing congestion from post-placed netlists. Similarly, in [40], different color schemes are used in post-placement images to represent routing channels, CLB spots, multipliers, memories, IO blocks, and connectivity matrices. Row 6 in Table 2 represents the feature sources, while row 7 represents the datasets. Although all the work discussed in this section can accurately estimate the routing congestion, almost all the estimation is done only for CLB levels. No work estimates routing congestion for DSPs or BRAM, even though they consume a significant amount of resources in modern designs. Also, none of the work is properly integrated with open-source or commercial implementation tools. Easy integration of the prediction models with existing physical implementation tools is still an open research direction.

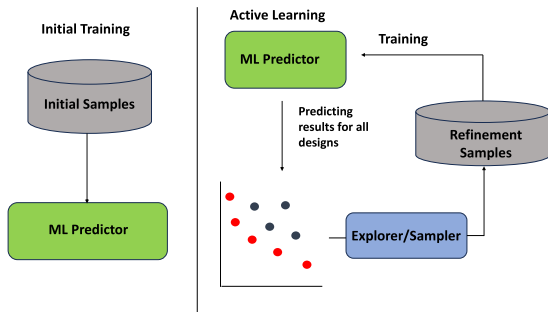


FIGURE 3. High level diagram of ML based DSE [36].

V. ML BASED DESIGN SPACE EXPLORATION OF HLS DESIGNS

In High-Level Synthesis, there are various methods that the designer can use to optimize their design in terms of area, latency, and timing. The process by which the designer achieve the best set of designs in term of area and latency by changing the parameters of the HLS tool is called “Design Space Exploration” (DSE). In HLS designs, there are three different ways known as “*knobs*”, which the designer can use to optimize their designs [76]. The three knobs of HLS designs are:

- 1) Pragma-based local synthesis
- 2) Global Synthesis Options
- 3) Functional Unit Constraints

The main challenge HLS users face is setting these knobs to obtain a design with the desired characteristics, considering that the total number of knob settings is extremely large. There are both traditional optimization methods like simulated annealing based, genetic algorithm based, ant colony optimization and gradient decent [33], [77], [78] based methods and machine learning based methods to perform DSE [33], [34], [35], [36], [79], [80]. Some of the works combine both traditional methods and ML-based methods [33]. Thus, the goal of HLS DSE is not to find a single solution but a set of designs, typically called Pareto optimal designs or microarchitectures, which form the Pareto frontier ($\bar{P}$). $\bar{P}$ can be defined as a set of dominating designs $\bar{P} = d_1, d_2, \dots, d_n$ such that for any design $d_i \in \bar{P}$; $A(d_i) \leq A(d_q)$ and $L(d_i) \leq L(d_q)$ [76]. There are twelve (Table 3) very recent works in which ML is applied to find the Pareto optimal curve in DSE for HLS. The researchers mostly use regression and transfer and active learning-based methods to perform DSE. In Fig. 3, we have shown the high-level flow of ML application to solve the DSE problem, which is common to most of the discussed papers. As shown in Fig. 3, most of the ML-based DSE tools initially synthesize a portion of the designs in real-time, which becomes the sample training dataset (shown in gray can in Fig. 3) and the rest are done using ML-based approach (shown in green can in Fig. 3).

A. DESIGN SPACE EXPLORATION USING STANDALONE ML

In [36] and [79], the authors use traditional ML algorithms to find the Pareto optimal curve. While [36], the authors have proposed a *transductive experimental design* (TED) based

learning method to speed up the design space exploration process, which can judiciously sample representative and hard-to-predict micro architecture choices and use them for training the learning model; in [79], the authors use ML to eliminate those points from the design space, which are almost certainly non-optimal. To do so, in [79] the authors propose a framework called Adaptive Threshold Non-Pareto Elimination (ATNE). Instead of improving the learner’s conventional accuracy, this work focuses on understanding and estimating the risk of losing “good” designs due to learning inaccuracy. Both these works use regression models to predict the area and latency requirement of the HLS designs and result in very fast exploration time. One drawback of [36] is they tested their model only on 242 designs, which is a very small subset of the actual design space of their tested design. Sohrabizadeh et al. [81] proposed a bottleneck-guided gradient optimizer to find a better design point later used in DSE systematically. They use a heuristic for ordering the parameters, which can eliminate the redundant design points, thereby reducing the design space significantly.

B. DESIGN SPACE EXPLORATION USING TRANSFER LEARNING

Transfer learning is widely applied in DSE of HLS. In transfer learning, the model is trained on one set of designs, and with minor modifications, the model can be deployed on another new set of designs for inference. In [35], [53] and [82], the authors apply transfer learning to move knowledge from a set of previously explored source designs to an unexplored target design. In [53], ML is not used, but the authors use a novel method, where mapping from the source design to the target design is done analytically for selecting the source design and the source directives. Here, the authors propose a code similarity matching model using string-similarity algorithms to quantify the similarity in terms of computational patterns that exist between a source design (from a library capturing prior knowledge) and the target design. After identifying similar designs, the next step is selecting the designs with similar pragma settings. The authors select the Pareto-optimal curve among the pragma settings in a source design and translate the Pareto-optimal curve from the source design to the corresponding ones in a target.

In [35], Kwon et al. apply transfer learning (TL) to speed up the DSE process in HLS designs. The main theme of this paper is to extract the knowledge obtained from the previously explored spaces of source designs and apply it to the early stage of DSE of a new target design. In [82], the authors train a set of DSE points on ASIC, and using ML, they predicted the performance of the same designs on FPGA. They have transformed the learning model from ASIC DSE to predict on FPGA DSE.

C. DESIGN SPACE EXPLORATION USING COMBINATION OF ML AND METAHEURISTICS METHODS

There are a few works [33], [34], which is a fusion of both metaheuristics-based methods like Simulated Annealing

TABLE 3. Machine learning based design space exploration works.

Paper	ML Model	QoR Metrics	Average ADRS	Dataset	Max. Design Space
[36]	TED+RF	ADRS	3.38	Custom DFT design	119
[35]	Transfer Learning	ADRS	4.27	Spector OpenCL	1173
[53]	Transfer Learning	ADRS	0.28	Machsuite	21952
[82]	Linear Regression	ADRS	27.47	S2Cbench	NA
[33]	Decision Tree	Run Time/Dominance	0.90 ^a	S2Cbench	NA
[83]	XGB	Prediction Accuracy	1.95 ^b	CNN architectures	744
[80]	XGB	ADRS	0.161	S2Cbench/Chstone	120,000
[34]	Decision Tree	ADRS	3.03	S2Cbench	NA
[79]	RF	Run Time	0.0155	Custom OpenCL BM	1527
[81]	Analytical	Run Time	5.04 ^c	Xilinx Vitis Library	NA
[26]	GNN	Resource/Latency	2.54x ^d /6x ^c	Machsuite/Poly/Chstone	4700(synth) /800(real)
[20]	Analytical	Run Time	215.12 ^c	Polybench	95

Note: ^a Average Dominance; ^b Average MAPE; ^c Average Speedup; ^d Average DSP Usage; NA: Not Available

(SA), genetic algorithm (GA), gradient descent, Ant Colony Optimization (ACO) etc. and machine learning models. In [33], the authors use decision trees to speed up the simulated annealing algorithm. Here [33], the authors use a decision tree to reduce the design space to speed up the DSE time and achieve comparable results to a non-pruned standard simulated annealing-based DSE. In the initial phases, standard simulated annealing generates designs based on the previous design's cost function. The initial designs are used to generate a decision tree to decide which of the attributes contributes to maximizing the cost function. The attributes from the decision tree, which shows high correlations, are fixed, while the less important attributes are selected pseudo-randomly. This considerably reduces the design space and allows finding the Pareto-front faster than a regular simulated annealer. In [34], the authors use ML to select the hyperparameters of the metaheuristic algorithms used in the DSE. Some of the meta-heuristic parameter examples selected in this work are the initial and final temperature, the descent rate, the exit condition in simulated annealing, and, in the GA case, the number of parent pairs and the mutation and crossover rate. The authors also proposed a combined SA, GA, and ACO concurrent multiheuristic design space explorer. In [80] Goswami et al. combined heuristics-based Simulated annealing and ML regression model to design a fast design space explorer in real designs. The initial part of the DSE algorithm runs on logic synthesis-based results, and once a sufficient amount of data (design points) has been generated, they create an ML model and switch to fast ML-based predictive DSE. They used this work to run DSE on different CNN architectures [83]. In [26], the authors created a fast DSE by applying GNN on LLVM IR graphs, which predicts a design's resource and latency requirements.

D. SUMMARY AND FUTURE DIRECTIONS FOR ML BASED DSE

One of the figures merits of measuring the performance of a DSE tool is called "Average Distance to Reference Set" (ADRS). This parameter measures how close the predicted DSE points are to the exhaustive search-based DSE points. If the value of ADRS is close to zero, it is a very

good design space explorer. The larger this value is, the worse the performance of the explorer. An explorer aims to minimize the ADRS or runtime. This is done by either minimizing the design space [35], [79] or by applying transfer learning from a different set of designs [35], [53] or from another platform like ASIC [82]. Most of the papers [34], [35], [36], [53], [80], [82] measure the performance of their DSE tool in terms of ADRS. In Table 3, column 3, we show what QoR measures are being used to measure the performance of the DSE tools as discussed in the papers, while in column 4, we reported the average ADRS values of the various designs used in each paper. For papers where QoR is measured using some other parameters like runtime, resource or latency requirements, the average of those values is reported. In columns 5 and 6, the benchmarks' names and the design space's maximum size are shown, respectively. Although there are few standard benchmarks like Chstone [84], Machsuite [85], Polybench [86] and S2Cbench [87] are available, most of the researchers try to generate their dataset [35], [36], [79], [81], [83]. Even if they use the existing benchmarks, they generate their own design versions. Hence, a fair comparison of the works is difficult to perform. Generating datasets to train ML models for PPA estimation or DSE is an extremely time-consuming process, which can take weeks and months. To address this issue, [88], [89], and [90] benchmarks have been created, specifically curated for ML training. These are large datasets similar to computer vision datasets like Imagenet [91], or Cifar10 [92], which are off-the-shelf datasets that the ML-based EDA researchers can directly use for training purposes. Another new direction of research is to eliminate the synthesis tools from the loop by using methods like LLVM [93] or MLIR [94] methods, which was used in [26] and [80].

VI. MACHINE LEARNING IN FPGA CAD TOOL PARAMETER SELECTION

As the technology node decreases, the CAD tools used to design the ICs are also becoming very complex. Many parameters are involved in EDA tools, which results in a huge design space. The runtime complexity of the CAD tools is also very large, and designs take multiple weeks to synthesize at each design point. Modern CAD tools used for logical

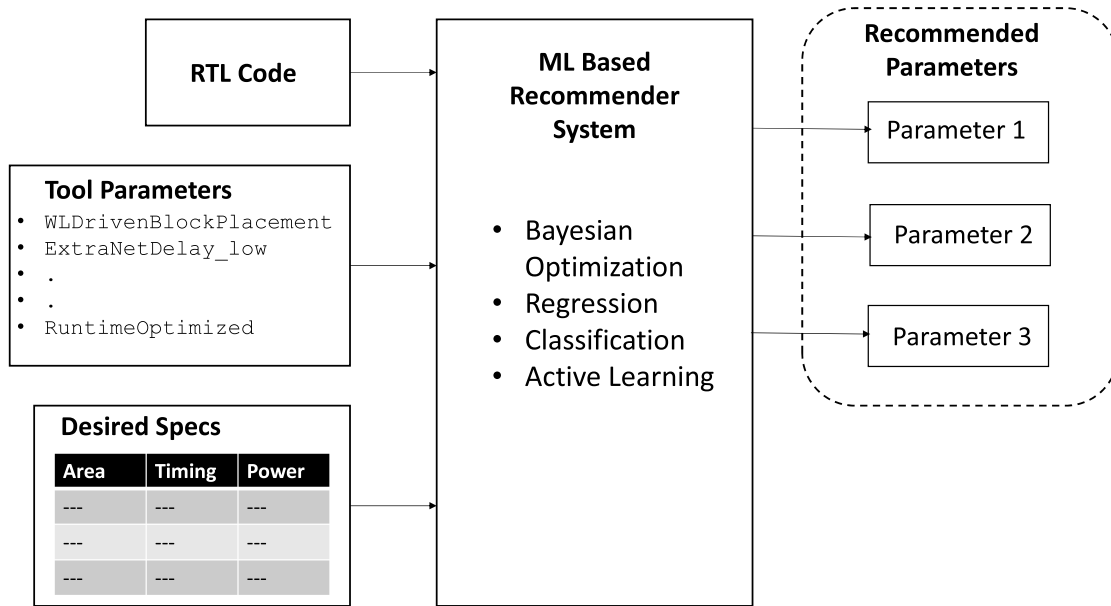


FIGURE 4. High level diagram showing how ML is used to recommend CAD tool parameters.

synthesis and physical design have hundreds of parameters in them, which are set to meet various timing and area requirements of the designs. The design space of these tools is humongous, and manually selecting the tool parameters will take ages and may not always generate the optimum design in terms of power, performance, and area. To recommend tool parameters and minimize design space in EDA tools, researchers have used Bayesian optimization [29], [30], [95], classification [28], [31], [54], Principal component analysis for design space pruning [95] and tensor decomposition and regression [32]. Researchers minimized the tools' runtime and generated area and timing-efficient designs using these methods. This section discusses seven recent works in which ML is applied to minimize search space in EDA tools and generate optimized designs. In Fig. 4, we have shown the high-level diagram of how ML can recommend CAD tool parameters for FPGA design. The input to the recommender system is generally the RTL code, a set of logic synthesis and physical design parameters that need to be tuned, and the desired specs in terms of area and performance. The recommender system considers the specs and the RTL code, and using ML, it suggests the best set of tool parameters to be used. All the works discussed in this section use a variant of this flow.

A. RECOMMENDER SYSTEM TO SUGGEST PARAMETERS TO MEET TIMING

The works discussed in [28], [31], and [55] propose methods to suggest CAD tool parameters to maximize total negative slack (TNS). InTime [28] is a plugin for FPGA CAD tools that can automatically select tool parameter assignments for each design by using machine learning heuristics and cheap cloud computing resources. While modern CAD tools have hundreds of parameters, InTime uses only 25 parameters

to suggest the best timing solutions. InTime is an iterative algorithm organized as a series of concurrent CAD runs. Each round, which consists of multiple concurrent runs, is an opportunity to generate candidate CAD parameter combinations and acquire data for analysis. Within each round, InTime uses a supervised learning approach to train classifiers that evaluate the effectiveness of a given combination of CAD parameter selections toward increasing timing slack.

Reference [31] is an extension of InTime [28]. In this paper, the authors proposed a method to select the best CAD tool parameters using a classification problem. They have minimized the search space using the Principal Component Analysis method. To train the model, they used the shelf ML models like Logistic regression (glm), Bagging (treebag), Random Forest (rf), Support Vector Machine (SVM) (svmRadial) and Neural Network (nnet). Here also, the authors minimized the clock period using the suggested CAD parameters.

Similar to [28] and [31], LAMBDA [55] also tries to maximize TNS. In [28] and [31], parameters of single stage are tuned, i.e. only for logic synthesis, placement or routing. But in [55], parameters of multiple stages are tuned simultaneously. They combine the features from multiple stages to predict post route QoR of the designs. They addressed the problem as a regression problem and used gradient boost regression to solve it.

B. GENERALIZED PPA RECOMMENDER SYSTEM

There are few other works [29], [30], [32], [54] which are much more generalized than those discussed in Section VI-A. These works suggest parameters not only to optimize timing but also to optimize other metrics like power, timing, and area. Reference [29] proposed a method that is used to

automate the flow selection in IC design using the Bayesian optimization method. Their optimization function optimizes a cost function consisting of power, performance, and area values. They automate the flow selection at logic synthesis and the place and route stage. They have used Gaussian process regression as the surrogate function for the Bayesian model. Using Bayesian optimization, the authors tuned six parameters, four of which are during the logic synthesis stage and two during the place and route stage. Similar to [29] and [32] also suggests tool parameters both at logic synthesis and place and route stage, using a two-step process. In the first stage, an ML model is trained offline. They have used macros and small partitions of large-scale, high-performance server processor chips for training. Once the model is trained, in the second stage, the tool inputs the macro name whenever a new macro comes. Suppose it is previously seen macro, set of cost functions to be optimized, or some baseline synthesized results for new macro and recommends the set of tool parameters to meet the objective function. Although the paper presents a nice idea about the recommendation system, nothing has been mentioned about the ML model or feature sets. Also, the paper does not discuss the accuracy of the resultant QoR of the recommendation system.

Reference [54] proposes a totally new approach compared to the already discussed ones. Instead of suggesting hyperparameters for CAD tools, this work suggests which will be the best tool to use to meet the specs of a particular design. They address the problem as a binary classification problem; for placement purposes, they selected two popular academic placers, gplace3.0 [75] and UTPlacef [74]. This work does not discuss the quality of the generated placed circuits in terms of timing, area, or power.

C. SUMMARY AND FUTURE DIRECTIONS

In Table 4, we have summarized the work done in the ML-based CAD tool parameter optimization domain. Except [29], all the other works are classification or regression. In the classification ones discussed in [28] and [31], the authors predict whether the QoR (area/timing, etc.) will be met or not met using a certain set of tool parameters. Similarly, in [54], a binary classification method is used to suggest which of the two academic placers is suitable to place and route a certain design; most of the work [28], [29], [31] work only at one stage of the physical design flow; i.e., either at synthesis, placement or routing. They cannot optimize parameters parallelly in multiple stages. This issue is addressed in LAMBDA [55], where they try to simultaneously autotune the hyperparameters in multiple stages, from logic synthesis to routing. Bayesian optimization is a popular hyperparameter optimization method which is widely used in ML frameworks to select optimal parameters. More research can be done in using Bayesian optimization methods for recommending automatic parameters in EDA flow at different stages.

TABLE 4. Summary of CAD tool autotuning works.

ML Method	Stage	Recommendation	References
Bayesian	LS, PR	Tool Parameters	[29], [30]
Classification	LS, PR	CAD flow	[54]
Classification	LS, PR	Tool Parameters	[28], [31]
Regression	LS, PR concurrent	Tool Parameters	[32], [55]

Note: LS: Logic Synthesis; PR: Place and Route

VII. MACHINE LEARNING TO PREDICT POWER, PERFORMANCE AND AREA ESTIMATION IN FPGA DESIGNS

Power, performance, and area estimation are crucial in any VLSI physical design flow. The earlier we predict the post-route QoR of design, the better it is so that the designer can go back and set the tool parameters to meet the specs. Few recent works use analytical or mathematical models or machine learning models to predict the post-route QoR of both HLS and RTL designs. Works like COMBA [20], LinAnalyzer [19] and Aladdin [21] use analytical methods to estimate the post route PPA of HLS designs, while “Fast and Accurate” [22], Pyramid [25], XPPE [23], HLSPredict [23] and Powergear [63] use ML-based methods to estimate the post route QoR of HLS designs. In [50] and [51] ML has been used to predict timing in RTL-based FPGA design, while in [62] and [63], GNN is used to predict power in FPGA designs..

A. ANALYTICAL METHODS FOR PPA ESTIMATION OF HLS DESIGNS

Aladdin [21], Comba [20], and Lin Analyzer [19] are three recent works that estimate power, performance, or area for HLS designs either individually or together using analytical and mathematical approaches without the need of any ML models. Aladdin [21] is a pre-RTL power performance simulator designed to enable rapid design space exploration of accelerator-centric systems. This framework takes high-level language descriptions of algorithms as inputs and uses dynamic data dependence graphs (DDDG) to represent an accelerator without generating RTL. Starting with an unconstrained program DDDG, which corresponds to an initial representation of accelerator hardware, Aladdin applies optimizations and constraints to the graph to create a realistic model of accelerator activity. Alladin generates different optimized DDDG graphs and applies various mathematical calculations to estimate the cycle-wise power, timing, and area requirement.

COMBA [20] is an analytical engine that is used to suggest pragmas during DSE. To generate the pragma recommendations, COMBA uses a database called recursive data collector (RDC) and a metric-guided design space exploration (MGDSE) algorithm. In input to the tool is a high-level HLS code written in C/C++. COMBA converts the HLS code into LLVM IR [73] codes and corresponding control and dataflow graphs (CDFG). By running analysis on the LLVM IR code and the CDFGs, they create a

mathematical model to estimate an FPGA design's latency and resource requirement. Based on the estimated resource and latency requirement, the *MGDSE* suggests pragmas for the next iteration. One major drawback of this work is that it compares their estimates against the estimation done by Xilinx Vivado HLS after the C-synthesis stage. Since the value reported by Vivado HLS is highly inaccurate [22], [25], [96], hence the estimation of COMBA is also wrong. However, their latency estimation is very good as compared to post-HLS estimation by the Vivado HLS tool.

B. ML BASED METHODS

In [22] and [25], the authors predicted post-route resource and timing requirements of HLS designs by using regression models based on post-synthesis log files. Generally, the QoR reported by an HLS tool after the C-synthesis stage varies significantly compared to post-route results. In [22], the authors analyzed the log files generated after the C-synthesis stage to extract various features to minimize the error. Later, they created regression models, which include Artificial Neural Networks, Linear Regression, and XGBoost, to predict the post-route area and performance of the designs. The work described in Pyramid [25] is very similar to [22]. Although the prediction results are good, a major drawback of this work is we have to run C-synthesis on the designs to extract features. Also, they did not report the error/accuracy on individual benchmarks. The details of the used features are also missing in the work. In [96], the authors created regression models to predict post-route resource, timing, and latency of HLS designs based on features from the high-level C++ code and LLVM IR codes, in [97] the authors constructed a regression model using four different regression models. They extracted the features from the post-high-level synthesis log files and the labels from a customized activity generator. They later used the model to create a design space explorer for latency vs power consumption and achieved very low ADRS values. However, the work does not mention the details at which stage the labels are generated. A random forest-based ML is explored to predict net delay in FPGA placement was proposed in [51]. Most of the features are extracted from the post-synthesized netlist, which includes fanout, Half Perimeter Wirelength (HPWL), congestion, critical path length etc. They later applied the Recursive Feature Elimination (RFE) method to reduce the number of features to 20 and used it to predict delay in RTL-based designs.

C. GRAPH LEARNING BASED METHODS

References [26] and [27] use graph learning methods to predict the post-route timing of HLS designs. In this work [27], the authors proposed a graph-based learning method to measure the delay of post-mapped arithmetic operations in HLS designs. They used Graph Neural Networks (GNN) to do so. Since GNNs can be used to predict the property of nodes and edges, they leveraged this property to predict the delay of mapped arithmetic operations. They convert the HLS code into LLVM IR and corresponding Control and Dataflow

Graphs (CDFG) to generate the graphs. They created a bunch of microcircuits from the CHstone [84] benchmark suite to test their model and achieved 72% reduction in root-mean-square error compared to operation delay estimation in Vivado HLS.

Ironman [26] is a unified HLS PPA predictor and DSE tool that uses ML in both stages. In the first stage, they use a graph neural network to estimate the LUT requirement and timing of the HLS designs. In the second stage, Reinforcement Learning is used to create the DSE. They also created an intermediate stage called Code Transformer (CT), which converts the high-level HLS code into CDFGs. In this work, they proposed a transformation methodology that maps the multiplication operation to LUT or DSPs in an optimized manner depending on the latency and resource requirement. They tested on eight real and synthetic benchmarks from CHstone and Machsuits benchmark suites and achieved less than 10% Mean Absolute Percentage Error (MAPE) for both resource and timing requirements. Even though they discussed a DSE, nothing has been mentioned about the ADRS values as compared to metaheuristics-based approaches like SA and GA. The strategy for DSE is also not properly discussed in the paper.

References [62] and [63] are two recent works in which GNN has been applied to predict power in RTL and HLS designs. In this work [63], the authors predicted early-stage power estimation in HLS design using GNN. They considered the impact of interconnects in power modeling to model their predictor accurately. Here, they have created their graph from the HLS design. The HLS design is converted into graphs, where the nodes are represented as operations, and node features indicate micro-architectures. The edges are interconnected, and edge features are represented as netlist activities. They later created a customized GNN to predict dynamic and total power in an HLS design. Traditionally, in RTL-based designs, power is estimated using gate-level simulation of the synthesized netlist, switching activity estimation (SAE), and input-output register toggle rates. This method is very slow and can take months. In [62], the authors used gate-level netlists and corresponding input port and register toggle rates over a power window from simulation as input as the training graph and the toggle rates from gate-level simulation as labels. They used the model to predict SAE, which can later be converted to power.

D. CROSS PLATFORM BASED METHODS

In [23] and [24] use a methodology in which training is done on one set of architecture/platforms but prediction is done on FPGAs only. Unlike RTL code, HLS code written in high-level languages like C/C++ can run on any architectures like CPU, GPU and microcontrollers. The authors leveraged this in [23] and [24]. In [23], the authors run multiple designs on different Xilinx FPGA devices and predict the performance or speedup on Arm A9 devices. To create the model, they have extracted features from the post-C-synthesis log files, which are very similar to "Fast and Accurate" [22]

TABLE 5. Summary of machine learning based power, performance and area prediction tools.

Paper	Machine Learning Model	Feature Source	Design Entry	Predicted Parameters	Average Error (%)	Dataset
[22]	XGB	C-synthesis files	HLS	LUT FF BRAM DSP Performance	4.05 3.8 0.2 2.5 6.4	Chstone/Machsuite /S2Cbench
[52]	Ensemble Transfer Learning	LLVM, Synthesis and PR Files	HLS	LUT FF BRAM DSP	11 12 2 ~0	Multiple ML kernels
[20]	Analytical	LLVM IR	HLS	Latency	0.5	Polybench
[19]	Analytical	DDDG	HLS	Latency	2.52	Polybench
[23]	ANN	C-synthesis files	HLS	Performance	5	Custom designs
[25]	RF	C-synthesis files	HLS	LUT FF BRAM DSP Performance	13.33 18.33 13.33 16.66 15.66	CHStone/ Machsuite/ S2CBench/Rosetta
[97]	CNN	C-synthesis & Switching Activities	HLS	Power	4.67	Polybench
[24]	RF	CPU subtraces	HLS	Latency Power	9.8 7.8	Polybench
[26]	GNN	LLVM IR CDFG	HLS	LUT DSP Performance	8.3 3.8 4.4	Chstone/Machsuite /Polybench
[27]	GNN	LLVM IR CDFG	HLS	Interconnect Delay	0.93	Machsuite
[96]	XGB	LLVM IR	HLS	LUT Performance Latency	10.32 6.29 10.22	Chstone/ S2Cbench
[63]	GNN	LLVM IR	HLS	Power	6.20	Polybench
[62]	GNN	Gate Level NL	RTL	Power	4.35	Custom designs
[51]	RF	Gate Level NL	RTL	Timing	6%	ISPD2016
[21]	Analytical	DDDG	RTL	Performance Power Area	0.9 4.9 6.6	GEMM design space

and Pyramid [25]. This model helps designers to create a heterogeneous system to divide which portion of a design to run on the FPGA part and which part to run on Arm part. The major drawback of this work is the authors did not consider the effects of pragmas on their HLS designs.

In [24], the power and performance of an HLS design running on FPGA is predicted by creating a ML model based on features generated after running the design on CPUs. HLSPredict ensures time synchronicity between the host CPU and the target FPGA accelerator by using sub-traces for model training: sub-traces are epochs of workload execution time in the form of CPU countermeasures for the host, and FPGA cycle counts for the target. The authors perform a detailed analysis of a design running on x86 CPU by identifying the CPU microarchitectural subsystems and correlating them with the post-route performance and power when the same design runs on FPGA.

E. SUMMARY AND FUTURE DIRECTIONS

In Table 5, we have summarized the works done to predict FPGA designs' PPA using analytical methods and ML-based models. Most of the work discussed in Table 5 predicts the post-route behavior of HLS designs. Models like XPPE, "Fast and Accurate" and Pyramid [22], [23], [25] rely on post-C-synthesis log files to extract features; hence they

are slower as compared to LLVM-based models like [20], [26], [27], and [96]. Most of the works discussed here use their design entry as HLS because of the fast synthesis, implementation time, and large availability of benchmarks. In the RTL-based works shown in Table 5 [21], [51], [62] they use features from synthesized gate-level netlists to predict a design's power and timing requirements. While most of the works discussed here predict either resource or timing or latency, the work in [96] and [98] is a comprehensive one that can predict all the four post route QoR metrics of a design viz., Latency, Resource Requirement, Timing, and Power. There is no work in which more advanced ML models like BERT, Transformer, or Autoencoders are being used to enhance and predict the performance of EDA tools. Using these modern techniques, post-route PPA estimations can be made faster and more accurate simultaneously. In a very recent work [99] (August 2023), the Large Language Model (LLM) has been used to generate Verilog code based on design description in pure English language. These codes can be synthesized on both FPGA and ASIC design tools. In another work, the authors present their work, which, for the first time, shows how to generate hardware security properties using LLMs automatically. They created their own BERT model, Hardware security-BERT, which can read SoC design documentation and generate pertinent hardware

security-relevant properties [100]. Even though these two works are not directly applied to predict PPA in FPGA CAD tools, these two are promising works to show how technology is advancing towards LLM and other language models.

VIII. MACHINE LEARNING IN FPGA LOGICAL AND PHYSICAL DESIGN FLOW

For decades, both FPGA and ASIC physical design has been dominated by evolutionary algorithms like genetic algorithm, simulated annealing, and gradient descent algorithms [101], [102], [103], [104], [105], [106], [107]. More recently, these heuristic-based algorithms are replaced by analytical solvers for improved timing [74], [75], [108], [109], [110]. Similarly, most routing algorithms use either heuristic-based or analytical methods [103], [104], [111], [112], [113], [114]. But similar to any other EDA algorithms discussed previously, the analytical and heuristic-based methods are very slow for large designs, taking days to run one design. A few works [115], [116], [117], [118] improve the runtime by invoking simulated annealing in multiple CPUs and GPUs. However, this work needs a very large runtime to converge a solution. To improve the runtime of simulated annealing or analytical methods, the researchers accelerate the SA or analytical placement using reinforcement learning (RL). Kamakoti and Bhatia's work [119] was one of the first works in which RL is combined with a Support Vector Machine (SVM) to speed up the FPGA place and route process. Recently, researchers combined heuristics and reinforcement learning (RL) in FPGA placement stage [56], [57], [58], [59]. In [120], the authors used a convolution neural network to improve placement, while RL has been applied to synthesis and routing in [61] and [121] respectively. In this section, we will discuss how ML is applied to the three crucial stages of FPGA physical design flow viz., (i) Synthesis, (ii) Placement, and (iii) Routing. In Fig. 5, we have shown a high-level common flow of how RL is applied in different FPGA/ASIC physical design stages. In RL-based IC design, a vector of performance features represents the state of a circuit. The RL agent takes this vector as the input, executes an action by changing the design parameter values, computes the reward, and then moves to the new state (S_{t+1}) (Fig. 5), if the reward has increased, else it explores different available actions otherwise. The reward is computed from the EDA environment results like simulated annealing or genetic algorithm (shown in the top box of Fig. 5) [44]. Most works discussed in this section use a variant of the flow shown in Fig. 5.

A. MACHINE LEARNING IN FPGA SYNTHESIS

ML has not been widely applied in the Synthesis or mapping stage of FPGA physical synthesis since these stages are too early to predict QoR down the flow. In [50], the authors proposed a methodology to predict the timing characteristics of a circuit right after the technology mapping stage. Similar to [51] discussed in Section VII, in [50], the authors extracted

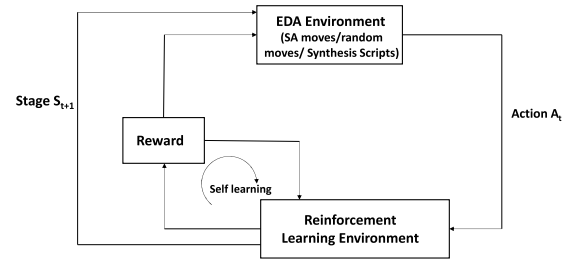


FIGURE 5. A generalized diagram of application of reinforcement learning in physical design.

features from the post-synthesized netlist. They used a gradient boost algorithm to train their regression model. However, they created the labels from the post-mapped netlist, not from the post-placed or post-route stage. However, this work is after the synthesis stage, right before mapping. There are few works in which RL is applied in Synthesis. In [121], Zhou and Anderson combined RL with ABC synthesis [122] to design a RL-guided synthesis optimization. They used eight different optimization parameters provided by `resyn2` script to define their action state space, and the reward function is calculated as the difference between the number of LUTs in the current state and the initial state (first stage). Among other RL-based synthesis works, Zhu et al. [123] propose the use of a graph convolutional network on a circuit AIG (And-inverter-graph) to leverage graph structural information for an RL agent to select which optimization commands to use. The action space of [123] is very similar to [121] based on ABC `resyn2`. Drills [124] is an RL-based design space explorer in the synthesis space where they used advantage actor-critic (A2C) to guide their RL agent. The reward function used in this work is to find a minimum design area subject to delay constraints.

B. MACHINE LEARNING IN FPGA PLACEMENT

Reinforcement learning has been widely used either standalone [60] or in combination with metaheuristic algorithms like simulated annealing [56], [57], [58], [59]. Most of these works use VTR [125] as their base placement algorithm and use RL to guide the annealer. In [56], Elgammal et al. combines simulated annealing (SA) with reinforcement learning. In SA, all the moves may not always result in improved timing or wirelength. The authors want their algorithm to “learn” the most effective perturbations using reinforcement learning RL. The RLPlace algorithm uses seven directed moves to optimize both wirelength and critical path delay. Here, RL guides the annealer to select the most effective moves throughout the placement to explore large placement solution space and achieve high-quality results quickly. This work [57] is very similar to [56] with several moves less than [56]. The proposed algorithm (directed moves + RL) explores the solution space more efficiently than traditional random moves, yielding better QoR for all tested designs. [58] is a precursor to [56] and [57] by Betz and his team. In [58], the authors combine SA and RL to guide the placement algorithm. But unlike [56] and [57],

only one state is used to guide the agent instead of using two state spaces at different temperatures of SA annealing. Also, in [56] and [57], seven directed moves were used for the annealer. In [58], only one move is selected: the Random Move in [56]. Here, they guide the RL agent based on four resource types in place of moves. Reference [59] is quite similar to [56], [57], and [60]. Here also, the authors applied RL to optimize moves of SA-based placer in VTR [101], [125]. But the details of the moves are not discussed in the work, nor are the results shown very prominently here.

In this paper [60], Malappa et al. applied RL in the detailed placement stage. They addressed the detailed placement problem as two staged hierarchical problems: (i) Coarse-grained refinement and (ii) Fine-grained refinement. They applied RL in Stage 1 to select the optimal sliding window size and the order in which the sliding windows must be rearranged. In Stage 2, Satisfiability Modulo Theories (SMT) are applied for fine-grain refinement. This work is purely RL-based, and they did not combine with any meta-heuristics-based methods like SA or Genetic Algorithm. In [120], the authors addressed the placement problem as a CNN problem based on electrostatic density. The authors proposed a CNN-inspired analytical global placement algorithm for large-scale FPGAs. A novel density framework was constructed to remedy the high computation time by casting the 2D electrostatic-based density constraints into CNN. This first and one of the only known works where CNN has been directly applied to the placement problem.

C. MACHINE LEARNING IN FPGA ROUTING

Although ML has been widely applied to predict congestion, routing violations, or DRC violations in routing, as discussed in Section IV, very few works are where ML is directly applied to guide the routing algorithm. RL has been applied to optimize the routing algorithm in one such work. In [61], the authors applied RL to optimize the routing algorithm and compared it against the conventional negotiated congestion routing method used in Pathfinder [111] routing algorithm. They used the cost function similar to [111] and optimized it using RL. However, the paper does not discuss the details of the RL algorithm and how the state table is created. This is the only work where the authors tried to apply ML in routing instead of merely prediction congestion or routing violation [37], [38], [41], [71].

D. SUMMARY AND FUTURE DIRECTION

In Table 6, we have summarized the application of ML in different stages of FPGA physical design flow. Reinforcement Learning is very similar to human learning. Just like humans learn from their mistakes and retrain themselves, RL also does so. Hence, RL is very suitable for physical synthesis optimization in EDA. Based on different reward functions, which are generally the cost functions of heuristic algorithms, the RL agent guides the optimizer. Hence, it is widely used in placement [56], [57], [59], [60] although application in routing and synthesis is limited. Future works in this direction

TABLE 6. Summary of ML based physical design tools.

Stage	ML Algorithm	Papers
Logic Synthesis	RL	[121], [123], [124]
	RL+GCN	[123]
	Regression	[50]
Placement	RL	[56]–[59]
	CNN	[120]
Routing	RL+GCN	[61]

may include generative AI like GAN, which will generate the placed circuit based on description like the requirement for timing and area. Just like GAN is used to create virtual congestion in [40] and [41], GAN can also be used for routing on the placed circuit in the future. In Table 6, we classified the three stages of physical design and what are the ML algorithms applied in each stage.

IX. CONCLUSION

ML is a growing field in the current technology domain, with many applications in computer vision, image processing, audio and video processing, NLP, etc. This field is currently being applied in the EDA domain. CAD-based IC design technology has been there since the 1980s, and we have lots and lots of data available at various stages of the physical design flow for various technology nodes. If we can utilize the data from past IC designs for ASIC and FPGA, we can make great tools using ML, resulting in fast IC design tools. Another challenge of ML-based IC design tools is the availability of specialists. The designer must have exceptionally good knowledge of VLSI design flow and ML technologies. As discussed in this chapter, almost all the available ML algorithms are now being applied in EDA tool design. However, most of the work discussed in this paper is still nascent in academic labs, and commercially making them available with vendor-supplied tools may take some time. Two recent tools from AMD Xilinx Vitis AI [126] and Synopsys DSO [127] use ML as part of their implementation algorithms.

REFERENCES

- [1] G. E. Moore, "Cramming more components onto integrated circuits," Reprinted from Electronics, volume 38, number 8, April 19, 1965, pp. 114 ff., *IEEE Solid-State Circuits Soc. Newslett.*, vol. 11, no. 3, pp. 33–35, Sep. 2006.
- [2] R. H. Dennard, F. H. Gaensslen, H. Yu, V. L. Rideout, E. Bassous, and A. R. LeBlanc, "Design of ion-implanted MOSFET's with very small physical dimensions," *IEEE J. Solid-State Circuits*, vol. SSC-9, no. 5, pp. 256–268, Oct. 1974.
- [3] P. Kannan, S. Balachandran, and D. Bhatia, "On metrics for comparing interconnect estimation methods for FPGAs," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 12, no. 4, pp. 381–385, Apr. 2004.
- [4] S. Balachandran and D. Bhatia, "A priori wirelength and interconnect estimation based on circuit characteristics," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 24, no. 7, pp. 1054–1065, Jul. 2005.
- [5] P. Kannan, S. Balachandran, and D. Bhatia, "fGREP—Fast generic routing demand estimation for placed FPGA circuits," in *Field-Programmable Logic and Applications*, G. Brebner and R. Woods, Eds. Berlin, Germany: Springer, 2001, pp. 37–47.
- [6] A. E. Caldwell, A. B. Kahng, S. Mantik, I. L. Markov, and A. Zelikovsky, "On wirelength estimations for row-based placement," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 18, no. 9, pp. 1265–1278, Sep. 1999.

- [7] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in *Proc. NIPS*, vol. 1, 2012, pp. 1097–1105.
- [8] S. Liu and W. Deng, "Very deep convolutional neural network based image classification using small training sample size," in *Proc. 3rd IAPR Asian Conf. Pattern Recognit. (ACPR)*, Nov. 2015, pp. 730–734.
- [9] T. P. Nagarhalli, V. Vaze, and N. K. Rana, "Impact of machine learning in natural language processing: A review," in *Proc. 3rd Int. Conf. Intell. Commun. Technol. Virtual Mobile Netw. (ICICV)*, Feb. 2021, pp. 1529–1534.
- [10] A. Le Glaz, Y. Haralambous, D.-H. Kim-Dufor, P. Lenca, R. Billot, T. C. Ryan, J. Marsh, J. DeVlyder, M. Walter, S. Berrouguet, and C. Lemey, "Machine learning and natural language processing in mental health: Systematic review," *J. Med. Internet Res.*, vol. 23, no. 5, May 2021, Art. no. e15708.
- [11] E. Mankolli and V. Guliashki, "Machine learning and natural language processing: Review of models and optimization problems," in *ICT Innovations 2020. Machine Learning and Applications*, V. Dimitrova and I. Dimitrovski, Eds. Cham, Switzerland: Springer, 2020, pp. 71–86.
- [12] J. Long, X. Wang, W. Zhou, J. Zhang, D. Dai, and G. Zhu, "A comprehensive review of signal processing and machine learning technologies for UHF PD detection and diagnosis (I): Preprocessing and localization approaches," *IEEE Access*, vol. 9, pp. 69876–69904, 2021.
- [13] Rahul, "Review of signal processing techniques and machine learning algorithms for power quality analysis," *Adv. Theory Simulations*, vol. 3, no. 10, Oct. 2020, Art. no. 2000118.
- [14] X. Dong, D. Thanou, L. Toni, M. Bronstein, and P. Frossard, "Graph signal processing for machine learning: A review and new perspectives," *IEEE Signal Process. Mag.*, vol. 37, no. 6, pp. 117–127, Nov. 2020.
- [15] (2019). *Scikit-Learn, ML in Python*. [Online]. Available: <https://scikit-learn.org/>
- [16] (2021). *An End-to-End Open Source Machine Learning Platform*. [Online]. Available: <https://www.tensorflow.org/>
- [17] (2021). *Pytorch*. [Online]. Available: <https://www.pytorch.org/>
- [18] X. Yang, R. Kastner, and M. Sarrafzadeh, "Congestion estimation during top-down placement," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 21, no. 1, pp. 72–80, Jan. 2002.
- [19] G. Zhong, A. Prakash, Y. Liang, T. Mitra, and S. Niar, "Lin-analyzer: A high-level performance analysis tool for FPGA-based accelerators," in *Proc. 53rd ACM/EDAC/IEEE Design Autom. Conf. (DAC)*, Jun. 2016, pp. 1–6.
- [20] J. Zhao, L. Feng, S. Sinha, W. Zhang, Y. Liang, and B. He, "COMBA: A comprehensive model-based analysis framework for high level synthesis of real applications," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Design (ICCAD)*, Nov. 2017, pp. 430–437.
- [21] Y. Sophia Shao, B. Reagen, G.-Y. Wei, and D. Brooks, "Aladdin: A pre-RTL, power-performance accelerator simulator enabling large design space exploration of customized architectures," in *Proc. ACM/IEEE 41st Int. Symp. Comput. Archit. (ISCA)*, Jun. 2014, pp. 97–108.
- [22] S. Dai, Y. Zhou, H. Zhang, E. Ustun, E. F. Y. Young, and Z. Zhang, "Fast and accurate estimation of quality of results in high-level synthesis with machine learning," in *Proc. IEEE 26th Annu. Int. Symp. Field-Program. Custom Comput. Mach. (FCCM)*, Apr. 2018, pp. 129–132.
- [23] H. M. Makrani, H. Sayadi, T. Mohsenin, S. Rafatirad, A. Sasan, and H. Homayoun, "XPPE: Cross-platform performance estimation of hardware accelerators using machine learning," in *Proc. 24th Asia South Pacific Design Autom. Conf.*, Jan. 2019, pp. 727–732.
- [24] K. O'Neal, M. Liu, H. Tang, A. Kalantar, K. DeRenard, and P. Brisk, "HLSPredict: Cross platform performance prediction for FPGA high-level synthesis," in *Proc. IEEE/ACM Int. Conf. Computer-Aided Design (ICCAD)*, Nov. 2018, pp. 1–8.
- [25] H. M. Makrani, F. Farahmand, H. Sayadi, S. Bondi, S. M. P. Dinakarrao, H. Homayoun, and S. Rafatirad, "Pyramid: Machine learning framework to estimate the optimal timing and resource usage of a high-level synthesis design," in *Proc. 29th Int. Conf. Field Program. Log. Appl. (FPL)*, Sep. 2019, pp. 397–403.
- [26] N. Wu, Y. Xie, and C. Hao, "IronMan: GNN-assisted design space exploration in high-level synthesis via reinforcement learning," in *Proc. Great Lakes Symp. (VLSI)*, Jun. 2021, pp. 39–44.
- [27] E. Ustun, C. Deng, D. Pal, Z. Li, and Z. Zhang, "Accurate operation delay prediction for FPGA HLS using graph neural networks," in *Proc. IEEE/ACM Int. Conf. Comput. Aided Design (ICCAD)*, Nov. 2020, pp. 1–9.
- [28] N. Kapre, H. Ng, K. Teo, and J. Naude, "InTime: A machine learning approach for efficient selection of FPGA CAD tool parameters," in *Proc. ACM/SIGDA Int. Symp. Field-Program. Gate Arrays*, Feb. 2015, pp. 23–26.
- [29] Y. Ma, Z. Yu, and B. Yu, "CAD tool design space exploration via Bayesian optimization," 2019, *arXiv:1912.06460*.
- [30] B. Reagen, J. M. Hernández-Lobato, R. Adolf, M. Gelbart, P. Whatmough, G.-Y. Wei, and D. Brooks, "A case for efficient accelerator design space exploration via Bayesian optimization," in *Proc. IEEE/ACM Int. Symp. Low Power Electron. Design (ISLPED)*, Jul. 2017, pp. 1–6.
- [31] Q. Yanghua, H. Ng, and N. Kapre, "Boosting convergence of timing closure using feature selection in a learning-driven approach," in *Proc. 26th Int. Conf. Field Program. Log. Appl. (FPL)*, Aug. 2016, pp. 1–9.
- [32] J. Kwon, M. M. Ziegler, and L. P. Carloni, "A learning-based recommender system for autotuning design Flows of industrial high-performance processors," in *Proc. 56th ACM/IEEE Design Autom. Conf. (DAC)*, Jun. 2019, pp. 1–6.
- [33] A. Mahapatra and B. C. Schafer, "Machine-learning based simulated annealer method for high level synthesis design space exploration," in *Proc. Electron. Syst. Level Synth. Conf. (ESLsyn)*, May 2014, pp. 1–6.
- [34] Z. Wang and B. C. Schafer, "Machine learning to set meta-heuristic specific parameters for high-level synthesis design space exploration," in *Proc. 57th ACM/IEEE Design Autom. Conf. (DAC)*, Jul. 2020, pp. 1–6.
- [35] J. Kwon and L. P. Carloni, "Transfer learning for design-space exploration with high-level synthesis," in *Proc. ACM/IEEE 2nd Workshop Mach. Learn. CAD (MLCAD)*, Nov. 2020, pp. 163–168, doi: 10.1145/3380446.3430636.
- [36] H.-Y. Liu and L. P. Carloni, "On learning-based methods for design-space exploration with high-level synthesis," in *Proc. 50th ACM/EDAC/IEEE Design Autom. Conf. (DAC)*, May 2013, pp. 1–7.
- [37] D. Maarouf, A. Alhyari, Z. Abuowaimer, T. Martin, A. Gunter, G. Grewal, S. Areibi, and A. Vannelli, "Machine-learning based congestion estimation for modern FPGAs," in *Proc. 28th Int. Conf. Field Program. Log. Appl. (FPL)*, Aug. 2018, pp. 427–427.
- [38] C.-W. Pui, G. Chen, Y. Ma, E. F. Y. Young, and B. Yu, "Clock-aware ultrascale FPGA placement with machine learning routability prediction: (Invited paper)," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Design (ICCAD)*, Nov. 2017, pp. 929–936.
- [39] J. Zhao, T. Liang, S. Sinha, and W. Zhang, "Machine learning based routing congestion prediction in FPGA high-level synthesis," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Mar. 2019, pp. 1130–1135.
- [40] C. Yu and Z. Zhang, "Painting on placement: Forecasting routing congestion using conditional generative adversarial nets," in *Proc. 56th Annu. Design Autom. Conf.*, Jun. 2019, pp. 26–31.
- [41] M. B. Alawieh, W. Li, Y. Lin, L. Singhal, M. A. Iyer, and D. Z. Pan, "High-definition routing congestion prediction for large-scale FPGAs," in *Proc. 25th Asia South Pacific Design Autom. Conf. (ASP-DAC)*, Jan. 2020, pp. 26–31.
- [42] G. Huang, J. Hu, Y. He, J. Liu, M. Ma, Z. Shen, J. Wu, Y. Xu, H. Zhang, K. Zhong, X. Ning, Y. Ma, H. Yang, B. Yu, H. Yang, and Y. Wang, "Machine learning for electronic design automation: A survey," *ACM Trans. Des. Autom. Electron. Syst.*, vol. 26, no. 5, pp. 1–46, 2021.
- [43] D. S. Lopera, L. Servadei, G. N. Kiprit, S. Hazra, R. Wille, and W. Ecker, "A survey of graph neural networks for electronic design automation," in *Proc. ACM/IEEE 3rd Workshop Mach. Learn. CAD (MLCAD)*, Aug. 2021, pp. 1–6.
- [44] R. Mina, C. Jabbour, and G. E. Sakr, "A review of machine learning techniques in analog integrated circuit design automation," *Electronics*, vol. 11, no. 3, pp. 1–20, 2022.
- [45] V. Hamolia and V. Melnyk, "A survey of machine learning methods and applications in electronic design automation," in *Proc. 11th Int. Conf. Adv. Comput. Inf. Technol. (ACIT)*, Sep. 2021, pp. 757–760.
- [46] A. B. Kahng, "Machine learning applications in physical design: Recent results and directions," in *Proc. Int. Symp. Phys. Design*, Mar. 2018, pp. 68–73.
- [47] M. Pandey, "Machine learning and systems for building the next generation of EDA tools," in *Proc. 23rd Asia South Pacific Design Autom. Conf. (ASP-DAC)*, Jan. 2018, pp. 411–415.
- [48] H. Ren, B. Khailany, M. Fojtik, and Y. Zhang, "Machine learning and algorithms: Let us team up for EDA," *IEEE Des. Test*, vol. 40, no. 1, pp. 70–76, Feb. 2023.

- [49] A. B. Kahng, "Machine learning for CAD/EDA: The road ahead," *IEEE Des. Test*, vol. 40, no. 1, pp. 8–16, Feb. 2023.
- [50] H. Hu, J. Hu, F. Zhang, B. Tian, and I. Bustany, "Machine-learning based delay prediction for FPGA technology mapping," in *Proc. 24th ACM/IEEE Workshop Syst. Level Interconnect Pathfinding*, Nov. 2023, pp. 1–6.
- [51] T. Martin, G. Grewal, and S. Areibi, "A machine learning approach to predict timing delays during FPGA placement," in *Proc. IEEE Int. Parallel Distrib. Process. Symp. Workshops (IPDPSW)*, Jun. 2021, pp. 124–127.
- [52] G. Singha, D. Diamantopoulos, J. Gómez-Lunaa, S. Stuijck, H. Corporaal, and O. Mutlue, "LEAPER: Fast and accurate FPGA-based system performance prediction via transfer learning," in *Proc. IEEE 40th Int. Conf. Comput. Design (ICCD)*, Oct. 2022, pp. 499–508.
- [53] L. Ferretti, J. Kwon, G. Ansaloni, G. Di Guglielmo, L. P. Carloni, and L. Pozzi, "Leveraging prior knowledge for effective design-space exploration in high-level synthesis," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 39, no. 11, pp. 3736–3747, Nov. 2020.
- [54] A. Al-hyari, Z. Abuowaimer, D. Maarouf, S. Areibi, and G. Grewal, "An effective FPGA placement flow selection framework using machine learning," in *Proc. 30th Int. Conf. Microelectron. (ICM)*, Dec. 2018, pp. 164–167.
- [55] E. Ustun, S. Xiang, J. Gui, C. Yu, and Z. Zhang, "LAMDA: Learning-assisted multi-stage autotuning for FPGA design closure," in *Proc. IEEE 27th Annu. Int. Symp. Field-Program. Custom Comput. Mach. (FCCM)*, Apr. 2019, pp. 74–77.
- [56] M. A. Elgammal, K. E. Murray, and V. Betz, "RLPlace: Using reinforcement learning and smart perturbations to optimize FPGA placement," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 41, no. 8, pp. 2532–2545, Aug. 2022.
- [57] M. A. Elgamma, K. E. Murray, and V. Betz, "Learn to place: FPGA placement using reinforcement learning and directed moves," in *Proc. Int. Conf. Field-Program. Technol. (ICFPT)*, Dec. 2020, pp. 85–93.
- [58] K. E. Murray and V. Betz, "Adaptive FPGA placement optimization via reinforcement learning," in *Proc. ACM/IEEE 1st Workshop Mach. Learn. CAD (MLCAD)*, Sep. 2019, pp. 1–6.
- [59] J. Zhang, F. Deng, and X. Yang, "FPGA placement optimization with deep reinforcement learning," in *Proc. 2nd Int. Conf. Comput. Eng. Intell. Control (ICCEIC)*, Nov. 2021, pp. 73–76.
- [60] U. Mallappa, S. Pratty, and D. Brown, "RLPlace: Deep RL guided heuristics for detailed placement optimization," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Mar. 2022, pp. 120–123.
- [61] U. Farooq, N. Ul Hasan, I. Baig, and M. Zghaibeh, "Efficient FPGA routing using reinforcement learning," in *Proc. 12th Int. Conf. Inf. Commun. Syst. (ICICS)*, May 2021, pp. 106–111.
- [62] Y. Zhang, H. Ren, and B. Khailany, "GRANNITE: Graph neural network inference for transferable power estimation," in *Proc. 57th ACM/IEEE Design Autom. Conf. (DAC)*, Jul. 2020, pp. 1–6.
- [63] Z. Lin, Z. Yuan, J. Zhao, W. Zhang, H. Wang, and Y. Tian, "PowerGear: Early-stage power estimation in FPGA HLS via heterogeneous edge-centric GNNs," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Mar. 2022, pp. 1341–1346.
- [64] N. Wu, H. Yang, Y. Xie, P. Li, and C. Hao, "High-level synthesis performance prediction using GNNs: Benchmarking, modeling, and advancing," in *Proc. 59th ACM/IEEE Design Autom. Conf.*, Jul. 2022, pp. 49–54.
- [65] J. H. Friedman, "Multivariate adaptive regression splines," *Ann. Statist.*, vol. 19, no. 1, pp. 1–67, Mar. 1991.
- [66] J. Elith, J. R. Leathwick, and T. Hastie, "A working guide to boosted regression trees," *J. Animal Ecol.*, vol. 77, no. 4, pp. 802–813, Jul. 2008.
- [67] M. Defferrard, X. Bresson, and P. Vandergheynst, "Convolutional neural networks on graphs with fast localized spectral filtering," 2016, *arXiv:1606.09375*.
- [68] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," 2016, *arXiv:1609.02907*.
- [69] K. Arulkumaran, M. P. Deisenroth, M. Brundage, and A. A. Bharath, "Deep reinforcement learning: A brief survey," *IEEE Signal Process. Mag.*, vol. 34, no. 6, pp. 26–38, Nov. 2017.
- [70] L. P. Kaelbling, M. L. Littman, and A. W. Moore, "Reinforcement learning: A survey," *J. Artif. Intell. Res.*, vol. 4, no. 1, pp. 237–285, Jan. 1996.
- [71] P. Goswami and D. Bhatia, "Congestion prediction in FPGA using regression based learning methods," *Electronics*, vol. 10, no. 16, p. 1995, Aug. 2021.
- [72] S. Yang, A. Gayasen, C. Mulpuri, S. Reddy, and R. Aggarwal, "Routability-driven FPGA placement contest," in *Proc. Int. Symp. Phys. Design*, Apr. 2016, pp. 139–143.
- [73] C. Lattner and V. Adve, "LLVM: A compilation framework for lifelong program analysis & transformation," in *Proc. Int. Symp. Code Gener. Optim. (CGO)*, Mar. 2004, pp. 75–86.
- [74] W. Li, S. Dhar, and D. Z. Pan, "UTPlaceF: A routability-driven FPGA placer with physical and congestion aware packing," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Design (ICCAD)*, Nov. 2016, pp. 1–7.
- [75] Z. Abuowaimer, D. Maarouf, T. Martin, J. Foxcroft, G. Gréwal, S. Areibi, and A. Vannelli, "GPlace3.0: Routability-driven analytic placer for UltraScale FPGA architectures," *ACM Trans. Design Autom. Electron. Syst.*, vol. 23, no. 5, pp. 1–33, Oct. 2018.
- [76] B. C. Schafer and Z. Wang, "High-level synthesis design space exploration: Past, present, and future," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 39, no. 10, pp. 2628–2639, Oct. 2020.
- [77] B. C. Schafer, "Parallel high-level synthesis design space exploration for behavioral IPs of exact latencies," *ACM Trans. Design Autom. Electron. Syst.*, vol. 22, no. 4, pp. 1–20, May 2017.
- [78] B. C. Schafer, "Probabilistic multiknob high-level synthesis design space exploration acceleration," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 35, no. 3, pp. 394–406, Mar. 2016.
- [79] P. Meng, A. Althoff, Q. Gautier, and R. Kastner, "Adaptive threshold non-Pareto elimination: Re-thinking machine learning for system level design space exploration on FPGAs," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Mar. 2016, pp. 918–923.
- [80] P. Goswami, B. C. Schaefer, and D. Bhatia, "Machine learning based fast and accurate high level synthesis design space exploration: From graph to synthesis," *Integration*, vol. 88, pp. 116–124, Jan. 2023.
- [81] A. Sohrabizadeh, C. H. Yu, M. Gao, and J. Cong, "AutoDSE: Enabling software programmers design efficient FPGA accelerators," in *Proc. ACM/SIGDA Int. Symp. Field-Program. Gate Arrays*, Feb. 2021, p. 147, doi: [10.1145/3431920.3439464](https://doi.org/10.1145/3431920.3439464).
- [82] S. Liu, F. C. Lau, and B. C. Schaefer, "Accelerating FPGA prototyping through predictive model-based HLS design space exploration," in *Proc. 56th ACM/IEEE Design Autom. Conf. (DAC)*, Jun. 2019, pp. 1–6.
- [83] P. Goswami, M. Shahshahani, and D. Bhatia, "Robust estimation of FPGA resources and performance from CNN models," in *Proc. 35th Int. Conf. VLSI Design 21st Int. Conf. Embedded Syst. (VLSID)*, Feb. 2022, pp. 144–149.
- [84] Y. Hara, H. Tomiyama, S. Honda, H. Takada, and K. Ishii, "CHStone: A benchmark program suite for practical C-based high-level synthesis," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, May 2008, pp. 1192–1195.
- [85] B. Reagen, R. Adolf, Y. S. Shao, G.-Y. Wei, and D. Brooks, "MachSuite: Benchmarks for accelerator design and customized architectures," in *Proc. IEEE Int. Symp. Workload Characterization (IISWC)*, Oct. 2014, pp. 110–119.
- [86] L.-N. Pouchet. (2020). *Polybench Benchmarks*. [Online]. Available: <https://web.cse.ohio-state.edu/~pouchet.2/software/polybench/>
- [87] B. C. Schafer and A. Mahapatra, "S2CBench: Synthesizable SystemC benchmark suite for high-level synthesis," *IEEE Embedded Syst. Lett.*, vol. 6, no. 3, pp. 53–56, Sep. 2014.
- [88] P. Goswami, M. Shahshahani, and D. Bhatia, "MLSBench: A synthesizable dataset of HLS designs to support ML based design flows," in *Proc. ACM/SIGDA Int. Symp. Field-Program. Gate Arrays*, Feb. 2020, pp. 1–6.
- [89] P. Goswami, M. Shahshahani, and D. Bhatia, "MLSBench: A benchmark set for machine learning based FPGA HLS design flows," in *Proc. IEEE 13th Latin Amer. Symp. Circuits Syst. (LASCAS)*, Mar. 2022, pp. 1–4.
- [90] Y. Zhou, U. Gupta, S. Dai, R. Zhao, N. Srivastava, H. Jin, J. Featherston, Y.-H. Lai, G. Liu, G. A. Velasquez, W. Wang, and Z. Zhang, "Rosetta: A realistic high-level synthesis benchmark suite for software programmable FPGAs," in *Proc. ACM/SIGDA Int. Symp. Field-Program. Gate Arrays*, Feb. 2018, pp. 269–278.
- [91] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2009, pp. 248–255.
- [92] N. V. Krizhevsky and G. Hinton, *The CIFAR-10 Dataset*. Toronto, ON, Canada: Univ. Toronto, 2009. [Online]. Available: <https://www.cs.toronto.edu/~kriz/cifar.html>
- [93] (2019). *LLVM Compiler*. [Online]. Available: <https://www.llvm.org>

- [94] (2020). *Multi-Level Intermediate Representation Overview*. [Online]. Available: <https://mlir.llvm.org/>
- [95] N. Kapre, B. Chandrashekar, H. Ng, and K. Teo, "Driving timing convergence of FPGA designs through machine learning and cloud computing," in *Proc. IEEE 23rd Annu. Int. Symp. Field-Program. Custom Comput. Mach.*, May 2015, pp. 119–126.
- [96] P. Goswami and D. Bhatia, "Predicting post-route quality of results estimates for HLS designs using machine learning," in *Proc. 23rd Int. Symp. Quality Electron. Design (ISQED)*, Apr. 2022, pp. 45–50.
- [97] Z. Lin, J. Zhao, S. Sinha, and W. Zhang, "HL-pow: A learning-based power modeling framework for high-level synthesis," in *Proc. 25th Asia South Pacific Design Autom. Conf. (ASP-DAC)*, Jan. 2020, pp. 574–580.
- [98] P. Goswami, "Machine learning based prediction in FPGA CAD," Ph.D. dissertation, Dept. Elect. Eng., Univ. Texas at Dallas, Richardson, TX, USA, May 2022.
- [99] S. Thakur, B. Ahmad, H. Pearce, B. Tan, B. Dolan-Gavitt, R. Karri, and S. Garg, "VeriGen: A large language model for verilog code generation," 2023, *arXiv:2308.00708*.
- [100] X. Meng, A. Srivastava, A. Arunachalam, A. Ray, P. H. Silva, R. Psiakis, Y. Makris, and K. Basu, "Unlocking hardware security assurance: The potential of LLMS," 2023, *arXiv:2308.11042*.
- [101] V. Betz and J. Rose, "VPR: A new packing, placement and routing tool for FPGA research," in *Proc. Int. Conf. Field-Program. Log. Appl.*, 1997, pp. 213–222.
- [102] J. M. Kleinhans, G. Sigl, F. M. Johannes, and K. J. Antreich, "GORDIAN: VLSI placement by quadratic programming and slicing optimization," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 10, no. 3, pp. 356–365, Mar. 1991.
- [103] W. Wang, Q. Meng, and Z. Zhang, "A survey of FPGA placement algorithm research," in *Proc. 7th IEEE Int. Conf. Electron. Inf. Emergency Commun. (ICEIEC)*, Jul. 2017, pp. 498–502.
- [104] S.-C. Chen and Y.-W. Chang, "FPGA placement and routing," in *Proc. IEEE/ACM Int. Conf. Computer-Aided Design (ICCAD)*, Nov. 2017, pp. 914–921.
- [105] G. Sergey, Z. Daniil, and C. Rustam, "Simulated annealing based placement optimization for reconfigurable systems-on-chip," in *Proc. IEEE Conf. Russian Young Researchers Electr. Electron. Eng. (EIconRus)*, Jan. 2019, pp. 1597–1600.
- [106] J. Yuan, J. Chen, L. Wang, X. Zhou, Y. Xia, and J. Hu, "ARBSA: Adaptive range-based simulated annealing for FPGA placement," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 38, no. 12, pp. 2330–2342, Dec. 2019.
- [107] P. Goswami and D. Bhatia, "Floorplanning of partially reconfigurable design on heterogeneous FPGA (abstract only)," in *Proc. ACM/SIGDA Int. Symp. Field-Program. Gate Arrays*, Feb. 2016, p. 275.
- [108] W. Li, Y. Lin, and D. Z. Pan, "ElPlace: Electrostatics-based placement for large-scale heterogeneous FPGAs," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Design (ICCAD)*, Nov. 2019, pp. 1–8.
- [109] C.-W. Pui, G. Chen, W.-K. Chow, K.-C. Lam, J. Kuang, P. Tu, H. Zhang, E. F. Y. Young, and B. Yu, "RippleFPGA: A routability-driven placement for large-scale heterogeneous FPGAs," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Design (ICCAD)*, Nov. 2016, pp. 1–8.
- [110] T. Liang, G. Chen, J. Zhao, S. Sinha, and W. Zhang, "AMF-placer: High-performance analytical mixed-size placer for FPGA," in *Proc. IEEE/ACM Int. Conf. Comput. Aided Design (ICCAD)*, Nov. 2021, pp. 1–9.
- [111] L. McMurchie and C. Ebeling, "PathFinder: A negotiation-based performance-driven router for FPGAs," in *Proc. 3rd Int. ACM Symp. Field-Program. Gate Arrays*, Feb. 1995, pp. 111–117.
- [112] J. Wang, J. Mai, Z. Di, and Y. Lin, "A robust FPGA router with concurrent intra-CLB rerouting," in *Proc. 28th Asia South Pacific Design Autom. Conf. (ASP-DAC)*, Jan. 2023, pp. 529–534.
- [113] K. E. Murray, S. Zhong, and V. Betz, "AIR: A fast but lazy timing-driven FPGA router," in *Proc. 25th Asia South Pacific Design Autom. Conf. (ASP-DAC)*, Jan. 2020, pp. 338–344.
- [114] M. Shen and G. Luo, "Corolla: GPU-accelerated FPGA routing based on subgraph dynamic expansion," in *Proc. ACM/SIGDA Int. Symp. Field-Program. Gate Arrays*, Feb. 2017, pp. 105–114.
- [115] Y. Lin, S. Dhar, W. Li, H. Ren, B. Khailany, and D. Z. Pan, "DREAMPlace: Deep learning toolkit-enabled GPU acceleration for modern VLSI placement," in *Proc. 56th ACM/IEEE Design Autom. Conf. (DAC)*, Jun. 2019, pp. 1–6.
- [116] A. Ludwin, V. Betz, and K. Padalia, "High-quality, deterministic parallel placement for FPGAs on commodity hardware," in *Proc. 16th Int. ACM/SIGDA Symp. Field Program. Gate Arrays*, Feb. 2008, pp. 14–23.
- [117] C. Fobel, G. Grewal, and D. Stacey, "A scalable, serially-equivalent, high-quality parallel placement methodology suitable for modern multicore and GPU architectures," in *Proc. 24th Int. Conf. Field Program. Log. Appl. (FPL)*, Sep. 2014, pp. 1–8.
- [118] M. An, J. G. Steffan, and V. Betz, "Speeding up FPGA placement: Parallel algorithms and methods," in *Proc. IEEE 22nd Annu. Int. Symp. Field-Program. Custom Comput. Mach.*, May 2014, pp. 178–185.
- [119] R. Manimegalai, E. Siva Soumya, V. Muralidharan, B. Ravindran, V. Kamakoti, and D. Bhatia, "Placement and routing for 3D-FPGAs using reinforcement learning and support vector machines," in *Proc. 18th Int. Conf. VLSI Design Held Jointly With 4th Int. Conf. Embedded Syst. Design*, Jan. 2005, pp. 451–456.
- [120] H. Wang, X. Tong, C. Ma, R. Shi, J. Chen, K. Wang, J. Yu, and Y.-W. Chang, "CNN-inspired analytical global placement for large-scale heterogeneous FPGAs," in *Proc. 59th ACM/IEEE Design Autom. Conf.*, Jul. 2022, pp. 637–642.
- [121] G. Zhou and J. H. Anderson, "Area-driven FPGA logic synthesis using reinforcement learning," in *Proc. 28th Asia South Pacific Design Autom. Conf. (ASP-DAC)*, Jan. 2023, pp. 159–165.
- [122] R. Brayton and A. Mishchenko, "ABC: An academic industrial-strength verification tool," in *Computer Aided Verification*. Springer, 2010, pp. 24–40.
- [123] K. Zhu, M. Liu, H. Chen, Z. Zhao, and D. Z. Pan, "Exploring logic optimizations with reinforcement learning and graph convolutional network," in *Proc. ACM/IEEE 2nd Workshop Mach. Learn. CAD (MLCAD)*, Nov. 2020, pp. 145–150.
- [124] A. Hosny, S. Hashemi, M. Shalan, and S. Reda, "DRiLLS: Deep reinforcement learning for logic synthesis," in *Proc. 25th Asia South Pacific Design Autom. Conf. (ASP-DAC)*, Jan. 2020, pp. 581–586.
- [125] K. E. Murray, O. Petelin, S. Zhong, J. M. Wang, M. Eldafrawy, J.-P. Legault, E. Sha, A. G. Graham, J. Wu, M. J. P. Walker, H. Zeng, P. Patros, J. Luu, K. B. Kent, and V. Betz, "VTR 8: High-performance CAD and customizable FPGA architecture modelling," *ACM Trans. Reconfigurable Technol. Syst.*, vol. 13, no. 2, pp. 1–55, Jun. 2020.
- [126] (2023). *Vitis AI*. [Online]. Available: <https://www.xilinx.com/products/design-tools/vitis/vitis-ai.html>
- [127] (2023). *Synopsys DSO*. [Online]. Available: <https://www.synopsys.com/ai/chip-design/dso-ai.html>



PINGAKSHYA GOSWAMI (Student Member, IEEE) received the B.Tech. degree in electronics and communication engineering from Tezpur University, India, in 2012, and the M.S. and Ph.D. degrees in electrical engineering from The University of Texas at Dallas, in 2016 and 2022, respectively. He is currently a Software Engineer with Lattice Semiconductor. His research interests include EDA tool development for FPGAs and ASICs and hardware accelerator design on FPGAs.



DINESH BHATIA (Senior Member, IEEE) received the bachelor's degree in electrical engineering from the Regional Engineering College, Suratkal, India, in 1985, and the master's and Ph.D. degrees in computer science from The University of Texas at Dallas, TX, USA, in 1987 and 1990, respectively. He is currently a Faculty Member with the Erik Jonsson School of Engineering and Computer Science, The University of Texas at Dallas, where he directs research activities within the IDEA Laboratory. He has served on technical program committees of several international conferences related to field-programmable gate arrays (FPGAs), field-programmable technology, and system-level design using FPGAs. His research interests include system-level design, power and energy systems, and architecture and computer-aided design for FPGAs. He has served on the Editorial Board of the IEEE TRANSACTIONS ON COMPUTERS. He was a Distinguished Lecturer of the IEEE Circuits and Systems Society, from 2007 to 2008.

...